Laboratorio 1 — Diseño del contrato .proto de un servicio gRPC

Proyecto: Plataforma de delivery (restaurantes, pedidos, repartidores, tracking) — Arquitectura de microservicios
Asignatura: Sistemas Distribuidos y Escalables | **Profesor:** Tomás Sarmiento Candia
Integrantes: Arturo Avalos y Benjamin Fernández

Servicio: **Order Service** Módulo Go: `github.com/oruava/order-svc` Fecha: 07/10/2026

Puerto: `:50051`

1. Contenido del archivo `pb/orders.proto`

```
syntax = "proto3";

package orders;

option go_package = "github.com/oruava/order-svc/pb";

import "google/protobuf/timestamp.proto";

// — Entidades ————————————————————————————————

// OrderItem: linea de un pedido (tabla order_items)
message OrderItem {
  string product_id = 1;
  int32 quantity = 2;
  double unit_price = 3;
  double subtotal = 4;
}

// ItemInput: producto + cantidad al momento de crear el pedido
message ItemInput {
  string product_id = 1;
  int32 quantity = 2;
}

// Order: entidad principal (tabla orders)
message Order {
  string order_id = 1;
  string client_id = 2;
  string restaurant_id = 3;
  string delivery_id = 4;  // opcional: se llena al asignar repartidor
  // status: CREATED | PAYMENT_PENDING | PAID | IN_PREPARATION |
  //         READY_FOR_PICKUP | ON_THE_WAY | DELIVERED | CANCELLED
  string status = 5;
  string delivery_address = 6;
  double total_amount = 7;
  repeated OrderItem items = 8;
  google.protobuf.Timestamp created_at = 9;
  google.protobuf.Timestamp updated_at = 10;
}

// — Mensajes de request/response ——————————————————

message CreateOrderRequest {
  string client_id = 1;
  string restaurant_id = 2;
  string delivery_address = 3;
  repeated ItemInput items = 4;
}
message CreateOrderResponse {
```

```
  Order order = 1;
}

message GetOrderRequest {
  string order_id = 1;
}
message GetOrderResponse {
  Order order = 1;
}

message UpdateOrderStatusRequest {
  string order_id = 1;
  string status = 2;
}
message UpdateOrderStatusResponse {
  string order_id = 1;
  string previous_status = 2;
  string new_status = 3;
  google.protobuf.Timestamp updated_at = 4;
}

// — Servicio ————————————————————————————————

service OrderService {
  rpc CreateOrder (CreateOrderRequest) returns (CreateOrderResponse);
  rpc GetOrder (GetOrderRequest) returns (GetOrderResponse);
  rpc UpdateOrderStatus (UpdateOrderStatusRequest) returns (UpdateOrderStatusResponse);
}
```

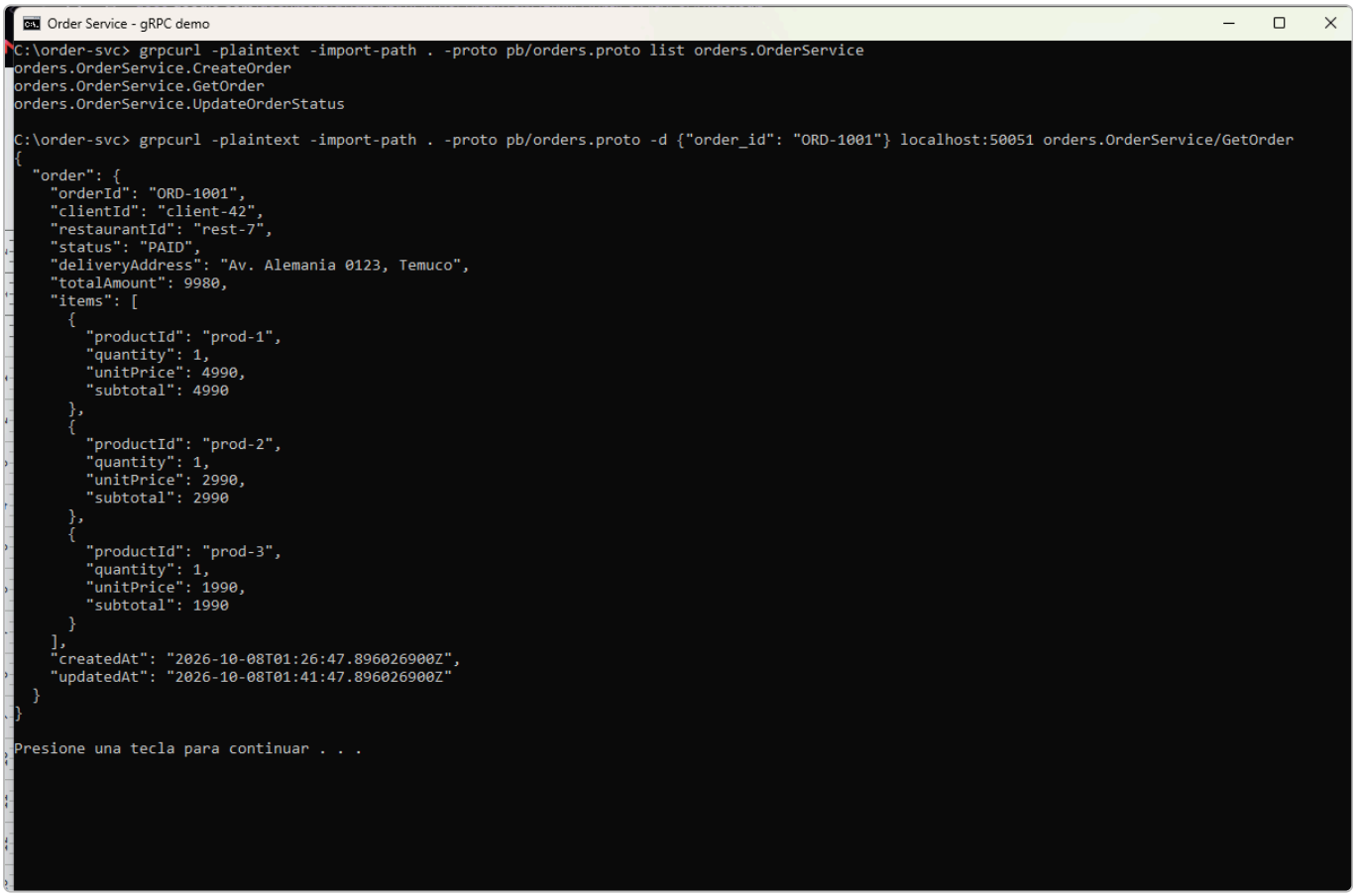
2. Prueba local con grpcurl

Paso 1 — Listar los métodos del servicio:

```
C:\order-svc> grpcurl -plaintext -import-path . -proto pb/orders.proto list orders.OrderService
orders.OrderService.CreateOrder
orders.OrderService.GetOrder
orders.OrderService.UpdateOrderStatus
```

Paso 2 — Llamada exitosa a `GetOrder`:

```
C:\order-svc> grpcurl -plaintext -import-path . -proto pb/orders.proto -d {"order_id": "ORD-1001"}
localhost:50051 orders.OrderService/GetOrder
```



Captura de pantalla: listado de métodos y llamada exitosa a orders.OrderService/GetOrder con grpcurl, respondiendo el pedido ORD-1001 con sus items, montos y timestamps.

3. Verificación del servidor

El servidor gRPC implementa los 3 métodos del contrato y responde en `localhost:50051`:

```
C:\order-svc> go run .
2026/10/07 22:43 servidor gRPC escuchando en :50051

# Build y código generado:
C:\order-svc> protoc --go_out=. --go_opt=paths=source_relative --go-grpc_out=. --go-
grpc_opt=paths=source_relative pb/orders.proto
```

```
C:\order-svc> go build -o order-svc.exe .   # BUILD OK

# Archivos generados:
pb/orders.proto          # contrato escrito a mano
pb/orders.pb.go          # tipos: Order, CreateOrderRequest, ...
pb/orders_grpc.pb.go     # interfaz del servidor y cliente gRPC
```